

Labo 7 - Architecture Événementielle : Pub/Sub et Event Sourcing

Cours : Architecture Logicielle (LOG430)

Session : Été 2025

Date du laboratoire : Semaine du 30 Juillet 2025



Le génie pour l'industrie

Contexte

À la suite du Labo 6 portant sur les sagas orchestrées, ce laboratoire introduit une approche décentralisée basée sur les événements. Dans une architecture événementielle, les microservices communiquent via un système de messagerie¹ selon le modèle *Pub/Sub*. Cette approche permet un fort découplage, une extensibilité accrue, et une meilleure résilience.

En complément, le **pattern Event Sourcing**² vous permettra de stocker l'historique complet des événements métier et de reconstruire l'état d'un agrégat à partir de ces événements.

Objectifs d'apprentissage

- Comprendre les principes de l'architecture événementielle (event-driven).
- Implémenter une communication Pub/Sub entre microservices.
- Gérer plusieurs abonnés (notification, audit...).
- Mettre en œuvre un Event Store persistant avec relecture d'événements.
- Instrumenter l'observabilité : métriques Prometheus, logs structurés, dashboards Grafana.
- Comprendre et expérimenter l'architecture CQRS (Command Query Responsibility Segregation) en lien avec Event Sourcing.

Tâches à réaliser

1. Définir un scénario métier

Choisissez un processus métier concret. Exemples :

- Traitement d'une réclamation client : création, affectation, traitement, clôture.
- Suivi d'un colis : enregistrement, expédition, livraison, incident.
- Panier e-commerce : ajout d'articles, paiement, expiration automatique.

Identifiez les événements clés (`ReclamationCreee`, `ColisLivre`, etc.)

2. Implémenter les producteurs d'événements

- Créez un ou plusieurs services qui publient des événements métiers.
- Utilisez un système messagerie comme RabbitMQ³, Kafka⁴ ou Redis Streams.
- Sérialisez les événements en JSON avec horodatage, ID unique, etc.
- Organisez les topics de manière claire (ex : `reclamations.events`).

1. Kafka : <https://kafka.apache.org/>, Redis Streams : <https://redis.io/docs/data-types/streams/>

2. Martin Fowler – Event Sourcing : <https://martinfowler.com/eaDev/EventSourcing.html>

3.

4. Tutoriel Kafka : <https://developer.confluent.io/quickstart/kafka-docker/>

3. Implémenter les abonnés aux événements

- Développez au moins deux services consommateurs :
 - Ex : notification utilisateur (email, Slack...).
 - Ex : enregistrement d’audit ou mise à jour d’état.
- Les abonnés doivent être idempotents et robustes.
- Loggez chaque événement traité dans un format structuré (JSON).

4. Mettre en œuvre un Event Store

- Enregistrez chaque événement dans une base (MongoDB, PostgreSQL, Redis, Kafka, etc.).
- Implémentez une fonctionnalité de *replay* pour reconstituer l’état métier.
- Offrez un endpoint pour afficher l’état courant d’un objet reconstruit.

5. Implémentation de CQRS avec Event Broker

- Séparez les responsabilités de commande (Command) et de lecture (Query) dans au moins un des services.
- Utilisez les événements publiés sur le bus comme point de synchronisation entre les projections de lecture et les actions métiers.
- Créez une ou plusieurs ”read models” à partir du Event Store ou des événements publiés (via projection, cache, indexation...).
- Les requêtes doivent interroger directement ces vues, sans impact sur la couche métier.
- Documentez cette séparation dans votre rapport (diagramme + justification).

6. Ajout d’observabilité

- Ajoutez des métriques dans vos services :
 - Nombre d’événements émis et consommés.
 - Latence d’un événement (émission → consommation).
- Utilisez Prometheus pour collecter les métriques.
- Créez un dashboard Grafana pour visualiser l’activité des topics.
- Tracez les événements dans les logs structurés avec contexte.

Livrables attendus

- Code source complet dans le dossier `lab7/`
- README.md avec instructions de déploiement
- Rapport synthétique (PDF ou Markdown) contenant :

- Le scénario métier retenu
- Le schéma de l’architecture événementielle (Pub/Sub + Event Store)
- Deux ADR (Architectural Decision Records)
- Exemples d’événements et relecture
- Captures d’écran des dashboards Grafana

Conseils

- Utilisez des noms de topics explicites et versionnés si nécessaire.
- Sérialisez les événements en JSON pour compatibilité.
- Ajoutez une clé de corrélation pour suivre les chaînes de traitement.
- Gérez les erreurs de consommation (logs, retry, dead-letter queue).
- Automatisez les tests de replay avec des cas d’usage réalistes.

1 Déploiement facultatif sur Kubernetes

Introduction

Ce volet est destiné aux étudiants souhaitant aller plus loin dans la mise en production et l’observabilité de leur architecture événementielle. Il est **facultatif**. Il vous permet de déployer votre solution dans un cluster Kubernetes (MicroK8s) hébergé à l’ÉTS.

Chaque étudiant pourra, sur demande (via Discord), recevoir un namespace dédié et les fichiers de connexion nécessaires (certificats et clés) pour interagir avec le cluster.

1.1 Étapes pour utiliser Kubernetes

1. Installer les outils requis

- **kubectl** : outil officiel en ligne de commande pour interagir avec un cluster Kubernetes.
- (*optionnel*) **OpenLens** : interface graphique simple pour naviguer dans vos pods, services, logs, etc.
- (*optionnel*) **Kompose** : pour convertir un fichier `docker-compose.yml` en manifestes Kubernetes.

2. Ajouter le cluster Kubernetes

Une fois les fichiers reçus (`ca.crt`, `cert.pem`, `key.pem`, `namespace.txt`), vous pouvez configurer votre cluster localement.

Listing 1 – Configuration du cluster (Windows)

```
kubectl config set-cluster log430 \
  —server=https://atreides.logti.etsmtl.ca:16443 \
  —certificate-authority="C:\chemin\vers\ca.crt" \
```

`—embed-certs=true`

```
kubectl config set-credentials student_XX \  
  —client-certificate="C:\chemin\vers\cert.pem" \  
  —client-key="C:\chemin\vers\key.pem" \  
  —embed-certs=true
```

```
kubectl config set-context student_XX@log430 \  
  —cluster=log430 \  
  —user=student_XX
```

```
kubectl config use-context student_XX@log430
```

Remarque : remplacez `student_XX` par votre identifiant et adaptez les chemins vers vos fichiers.

3. Tester la connexion à votre namespace

```
kubectl get pods -n <votre-namespace>
```

Votre namespace est contenu dans le fichier `namespace.txt`. Exemple PowerShell :

```
kubectl get pods -n $(Get-Content .\namespace.txt)
```

4. Déployer vos services avec YAML

Déployez vos producteurs, consommateurs et votre Event Store dans Kubernetes à l'aide de fichiers YAML (Deployment, Service, ConfigMap, Secret...).

- Les fichiers doivent être versionnés dans un dossier `kubernetes/` dans votre dépôt.
- Utilisez un **Service** de type `NodePort` pour exposer un service vers l'extérieur (Kafka UI, votre Event Store HTTP, etc.).
- Vérifiez les logs via :

```
kubectl logs <nom-du-pod> -n <votre-namespace>
```

1.2 Pour aller plus loin (optionnel)

- Utilisez **Grafana/Prometheus** pour monitorer les événements produits ou consommés.
- Déployez un Ingress Controller (déjà disponible via Nginx dans le cluster).
- Utilisez **Helm** pour simplifier les déploiements complexes.
- Effectuez des tests de charge (Artillery, Locust...) depuis un pod dans votre namespace.
- Activez des métriques personnalisées pour suivre le nombre d'événements ou les erreurs de traitement.

N.B. Toute demande d'accès au cluster doit être faite via Discord. Vous recevrez un dossier contenant vos identifiants et certificats d'accès personnalisés.